Ontology-Based Scope Control for Conversational Agents: A Comparison with Prose and Blocklist Baselines

Sravya Daggumalli
CSC789 AI Capstone
Dakota State University
Madison, SD, USA

***Abstract*—LLM-based conversational agents are usually told what they may respond to through a free-text system prompt, most often a list of forbidden topics. Such prompts cannot be checked for contradictions, miss cases their author did not anticipate, and do not let a permission on a broad category carry down to the narrower cases inside it. This project tests whether an ontology in which permissions are attached to concepts and inherited through the hierarchy makes better allow/refuse decisions for a city 311 assistant than a prose prompt, a blocklist, or a flat topic list, on the same set of questions. A pilot evaluation of the two baselines on 42 questions shows them failing in opposite directions: the flat list refuses nothing in scope but lets near-miss questions through, while the prompt catches near misses but refuses hazardous-but-routine requests as emergencies. On a frozen 154-question test split, the ontology reaches 0.94 accuracy when the hosted model is used only to name the concept and the ontology tree makes the decision, compared to 0.78 for the prompt with a blocklist (McNemar $p < 0.001$). While the blocklist cut down on prompt leaks, it doubled the wrong refusals. Embedding similarity also failed to separate near misses whether flat or hierarchical, and on questions that depend on inheritance, the ontology runs scored 0.97 to 1.00 against 0.61 to 0.73 for the prompts.**

***Index Terms*—conversational agents, guardrails, ontology, OWL, scope control, LLM**

I. Introduction

Today it is common practice to tell LLM-based conversational agents what they may respond to through a free-text system prompt, and those instructions are usually a list of things the agent must not do. I ran into three problems with this while building agent guardrails on Azure AI Foundry for a personal project. There is no good way to check a prose prompt for contradictions or gaps except by trying questions one at a time. A blocklist can miss related cases its author did not anticipate. And permissions do not inherit: allowing a broad category does not reliably allow the narrower cases inside it unless each one is spelled out. Vendor guardrails cover fixed harm categories like hate or self-harm, not what a specific agent is supposed to be *for*.

This project analyses whether a structured scope specification, an ontology where permissions are attached to concepts and a permission on a parent concept carries down to its children, makes better allow/refuse decisions than a prose prompt or a blocklist, and what it costs to build it. The point is not the use of an ontology by itself but the comparison: whether hierarchy and inheritance give better conversational scope control than prompt-based and flat approaches on the same questions.

II. Background and Related Work

Table I summarizes the eight papers reviewed. They fall into three groups.

A. Runtime policy enforcement for tool-using agents

The most active recent work puts a policy layer between an LLM agent and the actions it takes. AgentSpec [1] writes rules as trigger, check, and enforcement, evaluated before each tool call and kept outside the model; it blocks over 90% of unsafe code executions in its benchmarks at millisecond overhead. FORGE [2] treats policy as a cross-cutting concern in the aspect-oriented sense: a weaver instruments the agent system and a reference monitor checks each action against Datalog rules over a dependency graph, with static checks for contradiction and redundancy. Agent Behavioral Contracts [3] bring design-by-contract to agents with preconditions, invariants, and recovery. Ray [4] asks what a gate in front of tool calls can guarantee at all and formalizes the trade-off between false blocks and misses. These systems share two things: they govern which tool an agent may call, not what it may say, and their rules are flat or Datalog, so a permission at a general concept does not carry down to specific cases. What I take from them is the placement of the policy outside the agent, applied at every decision point.

B. Staying on topic in dialogue

A second group treats scope as a dialogue problem. NeMo Guardrails [5] lets developers write topical rails: a user message is mapped to a canonical form by embedding similarity and a dialogue flow decides what happens next. The topics are a flat list matched by similarity, which is what my flat-list baseline is. CantTalkAboutThis [6] fine-tunes the model on synthetic dialogues with distractor turns across nine domains; its human-written distractors turn out much harder than the synthetic ones (GPT-4-turbo reaches F1 0.68 on them). ENSToM [7] keeps small models on topic without training by steering activations at inference, raising distractor accuracy on LLaMA-2-7B from 0.28 to 0.81. These papers measure

the same two errors I measure, off-topic questions answered and on-topic questions refused. The difference is where the scope lives: inside the model weights or activations in the last two, with nothing to inspect or check, or as a flat list with no inheritance in NeMo.

C. Ontology-based agent policy before LLMs

The idea of expressing agent policy in OWL goes back to the early days of semantic web technologies. KAoS [8] represented authorizations and obligations as OWL classes, used a reasoner to find which policies applied to an action, and detected and resolved conflicts between policies, with lookups under a millisecond in deployments of over a thousand agents. Rei [9] took a similar approach for pervasive computing. This work targeted the software agents of its time and was never compared against prompt-based control, since LLM agents did not exist. It is the approach mine builds on. The KAoS authors also noted that OWL-DL alone could not express some policies they needed, which is one reason I keep the permission lookup in a small procedural step and use the reasoner for hierarchy consistency and conflict detection.

D. Gap and contribution

The first group enforces actions, not conversation. The second handles conversation but either inside the model or as a flat list. The third has hierarchy and conflict detection but predates LLMs and was never measured against prompting. In the papers I reviewed I did not find an empirical comparison of a hierarchical, checkable scope specification against prompt-based control for conversational scope. That is what this project does: the same questions, the same embedding model, a prose prompt, a flat list, and an ontology with inherited permissions, so that the effect of the hierarchy can be measured on its own.

III. Research Questions

Q1. Does an ontology with inherited permissions let fewer out-of-scope questions through than a prose prompt, a blocklist, or a flat topic list?

Q2. Does it do that without wrongly refusing more in-scope questions?

Q3. When it fails, is the failure in matching the question to a concept or in the policy itself?

IV. Project Challenges

The hardest part is matching a user's question to the right concept in the ontology. If the match is wrong, the reasoning on top of it cannot help, so I will measure this by running the ontology once with automatic matching and once with the correct concept supplied from my labels. When nothing matches, the system refuses, which is right for out-of-scope questions but wrong for in-scope ones, and I will report how often that happens. The prose and blocklist baselines need to be strong for the comparison to mean anything, so I will tune them on a small held-out set before the test set is fixed. Finally, the Dallas export I have covers only one department, so I pull the full request-type list through an aggregate query on the open data portal.

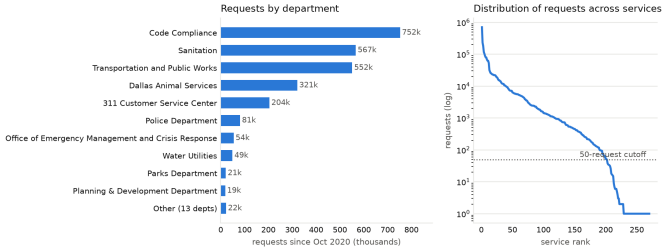


Fig. 1. Dallas 311 requests by department (left) and across all services sorted by volume, log scale (right).

V. Dataset

I use the Dallas 311 Service Requests open data, October 2020 to present [10], in aggregate only: one grouped query returns department and request-type counts (435 rows, about 2.6 million requests). After stripping department codes from the type names, merging duplicate department labels, and dropping test records, 270 services remain across 23 departments, 201 with at least 50 requests (Fig. 1). No individual records or locations are used.

The pilot evaluation set has 42 hand-written questions in four groups: direct (11), inheritance-dependent (10), paraphrase (10), and out-of-scope (11), each labeled allow or refuse with its concept. The out-of-scope group is built from near misses that reuse 311 vocabulary. The full set is described in the implementation section.

VI. Proposed AI Approach

Each approach takes a question and returns **allow** or **refuse** with a short reason. No chat interface or answer text is built; the decision is what is evaluated. Approaches A, B and R serve as the baselines.

- **(A) Prose prompt.** A system prompt describing what the assistant covers, run through a small hosted LLM.
- **(B) Prose + blocklist.** Same prompt plus an explicit list of forbidden topics.
- **(R) Flat list.** The same concepts as the ontology, no hierarchy, matched by embedding similarity. No LLM.
- **(C) Ontology (proposed).** generated from the Dallas request types (39 concepts under department classes), checked in Protégé, each branch marked permitted or denied.
  - Match the question to the closest concept with a small sentence-embedding model (`bge-small-en-v1.5`), or with the hosted LLM naming one concept from the list; no match means refuse.
  - Walk the concept's ancestors in Python with a most-specific-wins rule, so a parent's permission covers its children unless a child overrides it.

  - An OWL reasoner via `owlready2` [11] keeps the hierarchy consistent and flags any concept marked both permitted and denied.
- **(D) Ontology with gold concept.** Same as C but the correct concept comes from my labels, skipping matching. A diagnostic upper bound, not a usable system.

VII. Evaluation Plan

- **Metrics**, per approach and per question group on the test split: out-of-scope questions allowed; in-scope questions refused; for the ontology, how often a question matched any concept and how often an acceptable one.
- **Comparison:** McNemar's test between approaches (all see the same questions), plus a table of which questions each approach got wrong.
- **Qualitative:** the inheritance path used for questions that depend on it; the reasoner catching a concept marked both permitted and denied before any question is run.
- **Expected result:** the ontology does best on inheritance-dependent questions, loses some on reworded questions to matching errors, and differs little on questions that name a service directly.
- **Schedule:** taxonomy, pilot questions, and ontology done; full question set and all runs done; final report and presentation in December. API should cost under $30; everything else runs on my computer.

VIII. Baseline Results

The primary baseline is the prose system prompt, which is what AgentSpec [1], FORGE [2], and NeMo Guardrails [5] all compare against and what practitioners deploy. The second is a flat list of concepts matched by embedding similarity, the same approach NeMo's topical rails use; it is built from the same concepts, embedding model, and threshold as the ontology run, so the only difference between the flat list and the ontology run is the hierarchy.

Both were evaluated on the 42-question pilot set (Table II, Fig. 2). The prompt model (gpt-6-luna) does not accept a temperature setting, so it was run three times and the majority verdict scored; 3 of 42 verdicts changed between runs. The flat list was run with two concept lists: the pilot list, whose descriptions I wrote by hand after the questions existed, and a second list whose descriptions are taken verbatim from the Dallas service names with no wording of my own. The second list is the fair comparison, because its descriptions were written without any knowledge of the test questions. A higher threshold (0.65) was also tried to see whether it separates near misses from in-scope questions.

IX. Discussion of Baseline Results

Main findings. The two baselines fail in opposite directions. The flat list refused none of the 31 in-scope questions but allowed 5 of the 11 out-of-scope ones, and every miss was a near miss that reuses 311 vocabulary: the neighbor's tree matched tree maintenance, the pothole in Plano matched pothole repair, the dog-bite lawsuit matched animal neglect.

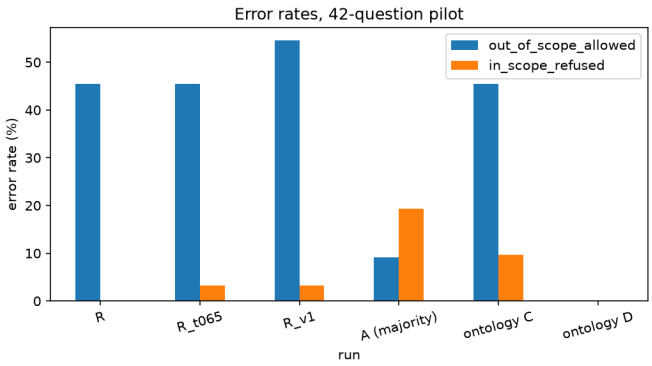


Fig. 2. Out-of-scope questions allowed and in-scope questions refused for each run, including the ontology with automatic matching (C) and with the gold concept (D). The flat list runs are those of Table II and the prompt is the majority of three runs.

Similarity matching gets the topic right and ignores location, intent, and who is responsible. The prompt, by contrast, allowed only the Plano pothole but refused 6 in-scope questions, and in every case its stated reason was that a hazard (a bent guardrail, a limb hitting trucks, a chained dog) was an emergency. Dallas 311 has request types for all of these. The same pothole with and without the words "cars keep hitting it" got opposite verdicts, so the wording triggers the refusal, not the topic. Replacing my hand-written concept descriptions with the city's own service names barely changed the flat list's verdicts, but 13 of 31 in-scope questions then matched the wrong concept; the verdicts held only because most concepts are permitted, so a wrong match still returned allow. Moreover raising the threshold to 0.65 did not separate near misses (scores 0.66 to 0.73) from real requests (down to 0.60). The prompt is also not repeatable due to non-determinism: 3 of 42 verdicts changed across three runs, and its reasons varied even when the verdict did not.

Limitations. This is a pilot on 42 questions, all written and labeled by me. The hand-written concept list was built after seeing the questions, which is biased for the flat list; the taxonomy-only list corrects this and is the one to be used from now on. The threshold was picked as starting value rather than tuned on a held-out split. One embedding model and one LLM were used, and three prompt runs is a small sample of its variability. The numbers just show us the direction for now not any meaningful differences statistically.

What should be improved. This was the plan at the pilot stage; the implementation section reports what was done. I have piloted with 42 questions but the real experiment needs to be conducted with a larger dataset. The questions I aimed initially is 160 with a tuning/test split, questions written by someone who is unaware of the ontology, plus a second labeler on a sample. The threshold should be tuned on the tuning split only and then frozen. For the ontology, concept accuracy matters more than verdict accuracy, since inheritance starts from the matched concept; with taxonomy-only descriptions it is 58%, so the ontology's labels need alternate names drawn

from the Dallas data rather than from the test questions. The pilot also showed two design requirements: hazardous-but-routine concepts must sit under permitted parents, separate from a denied Emergency concept so that they inherit permission, and denied concepts such as OtherJurisdiction and LegalMatter must be matched explicitly so near misses are refused with a valid reason. Finally, the ontology should be reported alongside the oracle condition with the gold concept supplied, to separate matching error from policy error.

X. Proposed Method: Implementation

Building the ontology. The ontology is generated by a Python script so it can be rebuilt at any time. The script reads the cleaned Dallas taxonomy (270 request types with department and count) which is the same taxonomy-only concept list used by the flat baseline, and an overrides file for hand-written policy exceptions, and writes the OWL file (`scope_v1.owl` for the pilot, `scope_v2.owl` after the v2 concept list). The tree has one root with two disjoint branches. `CityServiceRequest` is tagged permit and holds one class per Dallas department (six in the pilot) and one leaf per concept, placed under the department where it belongs. `NotHandled` is tagged deny and holds Emergency, LegalMatter, PrivatePropertyDispute, OtherJurisdiction, and OtherAgency. The pilot ontology contains 46 classes (Fig. 3). Permission is defined as an OWL annotation property attached only to the two branch roots (`permit` and `deny`); every other class has no tag of its own and inherits the permission of its nearest tagged ancestor. When the Dallas taxonomy includes a hazardous variant of a routine service (such as *Pot Hole - Hazardous*), it is modeled as an untagged `Hazardous` subclass under that service so that it automatically inherits `permit`. Exceptions are listed in the overrides file, one is added for the pilot: `AnimalAttackNow` sits under the permitted animal services branch but carries an explicit `deny` tag, since an active attack is a 911 call and Dallas has no request type for it.

Checking the policy at startup. Annotations are ignored by OWL reasoners by design, so the check has two parts. When the file is loaded initially, HermiT is run once using `owlready2`; because the two branches are disjoint, any class that ends up under both is inferred equivalent to `owl:Nothing` and is reported back. A second pass in Python reads the annotations and reports any class that carries both permit and deny, or a tag that is neither. If either part reports a problem the program stops and no question is answered. To verify the check works, an option flag (`plant`) plants five errors: a permitted concept placed under `NotHandled`, a denied concept placed under `CityServiceRequest`, a service placed under `LegalMatter`, a class with both tags, and a class tagged *maybe*. The check, as intended, reports six problems (Fig. 4); the extra one is the Hazardous child of the misplaced concept, which inherits both parents and is reported.

Deciding a question. At runtime the decision is made by a walk up the tree. Given a concept, the code looks at the concept and then its ancestors and returns the first permission

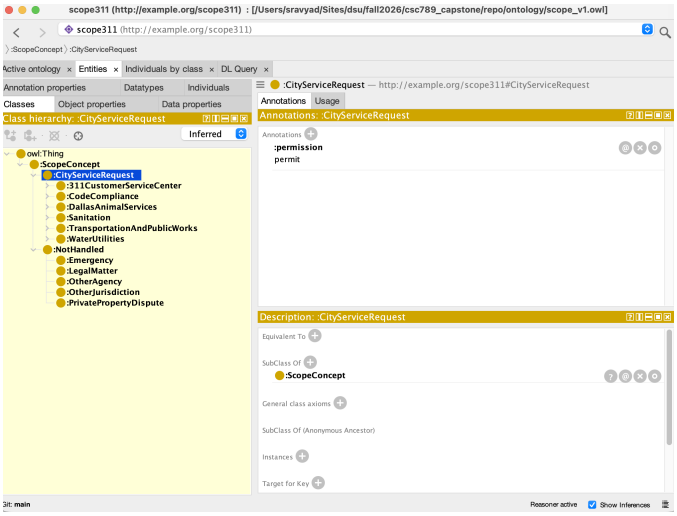


Fig. 3. Inferred class hierarchy of `scope_v1.owl` in Protégé after running HermiT reasoner. Every concept is satisfiable.

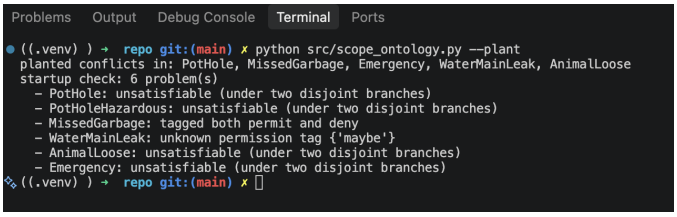


Fig. 4. Startup check with five planted policy errors. Six problems are reported; PotHoleHazardous is caught through inheritance.

found; a concept with no tagged ancestor is denied. The reasoner is not called per question but only at startup. Run D takes the gold concept from the question file, so it tests only the tree and its tags. Run C embeds every ontology class that has a description with the same sentence-embedding model and threshold as the flat baseline, takes the nearest class, and passes it to the same walk.

Pilot results. Both runs were scored on the 42-question pilot set (Table III, Fig. 2). Run D decides all 42 questions correctly and reaches the right concept for all 31 in-scope questions; the one question with no concept (weather) is unmatched and refused. This is expected for run D and confirms that the generated tree encodes the intended policy, including the inherited verdicts for the hazardous subclasses and the denied override.

With automatic matching (run C) the ontology decides 34 of 42 correctly, about the same as the flat list. The errors are almost all matching errors. Six in-scope questions land on the Hazardous subclass of the right concept, which still inherits permit and is decided correctly; counting those, concept accuracy is 18/31, equal to the flat list. The out-of-scope questions that were allowed are the same ones the flat list allowed, because the word *hazardous* in the generated descriptions attracts emergency wording such as a gas smell or a car on fire. The one question the hierarchy fixed is the dog attack with

an injured person, refused through the `AnimalAttackNow` exception; the flat list had allowed it as a loose animal. The same exception also refuses two in-scope animal questions (a chained dog without water, a stray dog chasing children), so it still needs a more detailed description. The gap between run C and run D, eight questions, is entirely the cost of matching, which the matching changes below address.

Concept list v2. The Dallas volume data showed that the 30 pilot concepts covered only 68% of requests and left out common services such as parking violations, utility line locate, encampments, noise, parks, animal bites and street lighting. Nine concepts were added with descriptions from the service names, and the ontology regenerated: 58 classes under nine department classes, 86% of requests. Baseline B, the prose prompt with an explicit twelve-item blocklist, was implemented with the same model and scoring as A; on the pilot it allowed 0/11 and refused 3/31, the same hazardous-but-routine refusals as A.

Evaluation set v1. The full set has 196 questions. The 42 pilot questions were written during the initial stages of the project and have been used in every design decision since, so they are included in the tuning split. 132 further questions were drafted as candidates from the Dallas service list ranked by request volume, so that every one of the 34 permitted concepts is asked about at least twice, then reviewed and labeled by me. 22 more were written independently by a family member from their own experience, without seeing the concept list; three of them concern things Dallas 311 does take (traffic calming, cut-through traffic, the outdoor siren) that the 39-concept policy does not model, and are labeled refuse under the policy as written. The 154 non-pilot questions form the test split, assigned once by a script with a fixed seed and committed before any test run. Nothing below was tuned on the test split. A second labeler on a sample is pending.

Improving the matching. Pilot run C showed three issues in the matcher for wrong concept selection:

- Department classes and hazardous subclasses were exposed as matching targets, which pulled queries away from their parent concept.
- The Dallas suffixes *Routine* and *Hazardous* in generated descriptions attracted emergency wording.
- The authored exception `AnimalAttackNow` was described too broadly and caught neglect and nuisance questions.

To fix this, I made three changes and verified on the 42 tuning questions only. First, the matcher now selects only among main concepts, while departments and hazardous subclasses remain in the tree for the permission walk. Second, the suffixes were stripped. Third, the exception was narrowed to an active attack in progress.

After these changes, concept accuracy on the tuning split improved from 14/31 to 17/31, while out-of-scope leaks stayed at 7/11. Two of these leaks matched on concepts added in v2 (parking ticket on `ParkingViolation` and bite lawsuit on `AnimalBite`). This indicates that a larger policy gives similarity matching more near-miss targets. Therefore, a second matcher was added using the same hosted model as baselines A and B to select one concept name from the list, while the tree still makes the decision. This keeps the model constant when comparing with B and only varies how the policy is represented. On the tuning split, it achieves 31/31 on concepts, 1/11 out-of-scope allowed, and 0/31 in-scope refused.

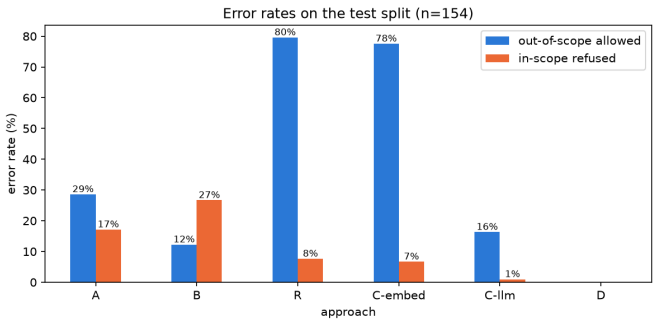


Fig. 5. Out-of-scope questions allowed and in-scope questions refused on the test split, all five approaches.

Results on the test split. Table IV and Fig. 5 compare all five approaches on the same 154 test questions (105 in-scope and 49 out-of-scope).

- **Overall accuracy:** The ontology with LLM matching (C-llm) is the most accurate, at 0.94 (8/49 out-of-scope allowed, 1/105 in-scope refused, and 96/105 correct concepts). In comparison, the prompt with blocklist reaches 0.78 and the plain prompt gets 0.79. While the blocklist reduced leaks from 14/49 to 6/49, it increased wrong refusals from 18/105 to 28/105. This trade-off was not visible earlier due to the small size of 31 in-scope questions in the pilot set.
- **Embedding similarity:** Both flat and hierarchical embedding similarity performed poorly on out-of-scope queries, allowing 39/49 and 38/49 respectively. Near misses are close to in-scope concepts by their nature, and adding the hierarchy does not change how similarity matching behaves.
- **Inheritance-dependent questions:** Grouping by question type, inheritance-dependent queries separate the approaches the most, where C-embed achieves 1.00, C-llm 0.97, R 0.94, A 0.73, and B 0.61.

For statistical significance, I ran McNemar's exact test between C-llm and each baseline, which gives $p < 0.001$ across all comparisons (for example, against B, C-llm gets 30 questions right where B is wrong, and only 5 in reverse). Against run D with the gold concept, it gives $p = 0.004$ due to the 9 remaining matching errors. Also, across three identical runs, A changed 14 and B changed 11 out of 154 verdicts, whereas the ontology tree decisions are deterministic. C-llm was evaluated in a single run, and three runs are planned before the final report.

What remains. A second labeler on a sample; three runs of the LLM matcher; the prompt conflict experiment (three contradictory items planted in the blocklist); the policy-scale

experiment (prompt against ontology at 100 and 270 concepts); the final report and presentation.

XI. What the Ontology Adds Beyond a Blocklist

Run D is perfect by construction and proves only that the tree encodes the policy; it is not evidence that the system works. The case for the ontology rests on properties a prose prompt or blocklist does not have, each of which was measured in this project.

Hard enforcement and determinism. A and B are soft controls: the model reads the policy and decides, and across three identical runs 14 and 11 of 154 test verdicts changed, with reasons that changed wording even when the verdict did not. R, C and D are hard controls: once a concept is chosen the verdict is a lookup, identical on every run, with no API call. Recent work frames prompt-based enforcement as having no guarantee and argues for deterministic mechanisms outside the model wherever the policy is concrete enough [12]; a 311 scope policy is exactly that concrete.

Exceptions as structure, not phrases. The hardest group is the inheritance-dependent one, where a hazardous pothole is an exception to *hazards are emergencies* and an attack in progress is an exception to *animal matters are allowed*. B refuses the bent guardrail and the limb hitting trucks because the emergency item of its list fires on a single phrase, which is the dominant failure the SafePyramid benchmark reports for in-context policies with exceptions [13]. In the ontology an exception is a node with a tag: `PotHoleHazardous` inherits permit from `PotHole`, `AnimalAttackNow` carries deny under a permitted department, and once the concept is right the verdict cannot over-trigger. Run D confirms this on every inherit question; run C shows what remains is recognizing the concept, a separable problem that B cannot separate.

Conflicts are caught before any question is asked. Five planted errors in the ontology produce six reported problems and the program refuses to start. The same kind of contradiction in a prompt is invisible; nothing reads a prompt for consistency. The most recent OWL-based policy work for LLM agents, by the Rei authors, lists static conflict detection as future work [14]; here it is a 40-line startup check.

Maintenance. Extending the policy from 30 to 39 concepts took nine rows in the concept file and no permission edits; inheritance supplied every verdict and the check confirmed the tree. The equivalent change to B is editing prose and rereading twelve blocklist items for interactions, with no way to confirm the result except rerunning questions.

An audit trail. Every ontology verdict records the matched concept and the class whose tag decided it (`decided_by`), so a wrong allow can be traced to a matching error or a policy error in one look. B's reason is a free-text sentence that varies between runs.

What it does not add. The hierarchy does not improve matching: the flat list and the ontology with the same embedding matcher are within one question of each other on every metric. The ontology's accuracy advantage comes from separating the two jobs, naming the concept and deciding the verdict, and giving each to the component suited to it. With that separation the ontology is ahead of the blocklist prompt by 16 points on the test split, the difference is significant, and the nine errors that remain against run D are all in matching.

XII. Expected Contribution

A small, reproducible comparison of prose, blocklist, flat-list, and hierarchical scope policies on a constructed 311 benchmark, released with the question set and ontology. It shows that the hierarchy with inherited permissions decides correctly whenever the concept is right, that automatic matching is where the error lives and becomes small once the matcher is an LLM, and that a structured policy can be checked for conflicts before use. The blocklist prompt did not match the ontology on the test split, and the comparison shows why: the blocklist bought fewer leaks with more wrong refusals.

AI use disclosure: AI tools were used to help format and edit sections of this report, to check references, and to assist with the data-preparation and baseline scripts, all of which I reviewed, modified, and tested. The design, ontology, question set, and analysis are my own.

References

[1] H. Wang, C. M. Poskitt, and J. Sun, "Agentspec: Customizable runtime enforcement for safe and reliable llm agents," in *Proceedings of the 2026 IEEE/ACM 48th International Conference on Software Engineering*, ser. ICSE '26. ACM, Apr. 2026, pp. 2938–2950. [Online]. Available: http://dx.doi.org/10.1145/3744916.3764546

[2] N. Palumbo, S. Choudhary, J. Choi, G. Amir, P. Chalasani, and S. Jha, "Formal policy enforcement for real-world agentic systems," 2026. [Online]. Available: https://arxiv.org/abs/2602.16708

[3] V. P. Bhardwaj, "Agent behavioral contracts: Formal specification and runtime enforcement for reliable autonomous ai agents," 2026. [Online]. Available: https://zenodo.org/doi/10.5281/zenodo.18775393

[4] S. Ray, "What can be enforced? a theory of certified runtime safety for tool-using agents," 2026. [Online]. Available: https://arxiv.org/abs/2607.22868

[5] T. Rebedea, R. Dinu, M. Sreedhar, C. Parisien, and J. Cohen, "Nemo guardrails: A toolkit for controllable and safe llm applications with programmable rails," 2023. [Online]. Available: https://arxiv.org/abs/2310.10501

[6] M. N. Sreedhar, T. Rebedea, S. Ghosh, J. Zeng, and C. Parisien, "Canttalkaboutthis: Aligning language models to stay on topic in dialogues," 2024. [Online]. Available: https://arxiv.org/abs/2404.03820

[7] H. Suh, Y. Jeon, D. Kang, T. Park, Y. Min, and G. G. Lee, "Enstom: Enhancing dialogue systems with entropy-scaled steering vectors for topic maintenance," 2025. [Online]. Available: https://arxiv.org/abs/2505.16526

[8] A. Uszok, J. Bradshaw, M. Johnson, R. Jeffers, A. Tate, J. Dalton, and S. Aitken, "Kaos policy management for semantic web services," *IEEE Intelligent Systems*, vol. 19, pp. 32–41, 08 2004.

[9] L. Kagal, T. Finin, and A. Joshi, "A policy language for a pervasive computing environment," in *IEEE 4th International Workshop on Policies for Distributed Systems and Networks*. IEEE, Jun. 2003, pp. 63–74.

[10] City of Dallas, "311 service requests, october 1, 2020 to present," Dallas OpenData, dataset d7e7-envw, 2026. [Online]. Available: https://www.dallasopendata.com/Services/311-Service-Requests/d7e7-envw

[11] J.-B. Lamy, "Owlready: Ontology-oriented programming in python with automatic classification and high level constructs for biomedical ontologies," *Artificial Intelligence in Medicine*, vol. 80, pp. 11–28, 2017. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0933365717300271

[12] N. Baracaldo, "From alignment to access control: A framework for genai policy enforcement," 2026. [Online]. Available: https://arxiv.org/abs/2609.26682

[13] J. Zhang, H. He, S. Zhang, S. Wang, X. Xu, Y. Sun, M. Shen, and F. Liu, "Safepyramid: A hierarchical benchmark for in-context policy guardrailing," 2026. [Online]. Available: https://arxiv.org/abs/2606.29887

[14] A. Joshi, T. Finin, K. Joshi, and L. Kagal, "Deontic policies for runtime governance of agentic ai systems," in *2026 IEEE International Conference on Web Services (ICWS)*. IEEE, July 2026, pp. 1255–1264.

TABLE I

RELATED WORK COMPARISON. P = PEER-REVIEWED; PRE = ARXIV PREPRINT. ROWS 1–4: RUNTIME POLICY ENFORCEMENT FOR TOOL-USING AGENTS; 5–7: STAYING ON TOPIC IN DIALOGUE; 8: ONTOLOGY-BASED AGENT POLICY BEFORE LLMS.

#	Paper	Problem	Dataset	Method	Baseline	Evaluation	Main finding	Limitation / relevance
1	AgentSpec, ICSE 2026 [1] (P)	LLM agents taking unsafe actions at runtime	RedCode-Exec (code), SafeAgentBench (embodied), FixDrive (driving)	Rules written in a small language: trigger, check, enforce; checked before each action, outside the model	Same agents with no rules	Share of unsafe actions blocked; runtime overhead; precision/recall of LLM-written rules	Blocks over 90% of unsafe code runs, all embodied hazards, all driving violations, at millisecond overhead	Rules are flat and act on tool calls; nothing about what the agent may or may not talk about. This is closest to my decision layer.
2	FORGE, arXiv 2026 [2] (pre)	Enforcing security and organizational policies across single and multi-agent systems	Prompt-injection tasks, τ^2-bench customer service, MALADE pharmacovigilance pipeline	Treats policy as a cross-cutting concern (aspect-oriented programming, AOP): an aspect weaver instruments the agent system, a reference monitor checks each action against Datalog rules over a causal dependency graph; static checks for contradiction, redundancy, subsumption	Same systems with the policy specified only in the prompt	Attack success, policy compliance, task success, latency and cost	Injection attacks 100% to 0%; τ^2-bench compliance 58% to 98%; latency up 19–38%	Enforces actions, not conversation. Its static checks are over rules, not a domain concept hierarchy. The weaver idea (policy outside the agent, applied at every decision point) and the contradiction check both can be mapped to my design.
3	Agent Behavioral Contracts, arXiv 2026 [3] (pre)	Agents drifting from intended behavior with no formal spec	AgentContract-Bench: 200 scenarios, 7 domains; 1,980 sessions on 7 models	Design-by-contract for agents: preconditions, invariants, governance, recovery; probabilistic compliance and a drift bound	Same agents without contracts	Violations detected per session, hard-constraint compliance, drift, recovery rate, overhead	Contracted agents catch 5–7 violations per session that baselines miss; 88–100% hard compliance; under 10 ms overhead	Contracts on actions and outputs; no domain scope model. Patent pending; code not available.
4	What Can Be Enforced?, arXiv 2026 [4] (pre)	What a runtime gate in front of tool calls can guarantee	AgentDojo task plans (132 items) scored by 23 judges; small controlled models	Theory: which policies a deterministic gate can represent; false-block vs. miss trade-off; conformal calibration; closed-loop effects	None (theory plus diagnostics)	Proofs; static diagnostic on judges; paired closed-loop reruns	Guarantees depend on what the gate can represent and observe; benign calibration does not survive representation attacks	Studies the same two errors I measure (wrongly allowed, wrongly refused), for tool calls rather than questions. Like the others, it covers tool calls only, not conversation.
5	NeMo Guardrails, EMNLP 2023 Demo [5] (P)	Keeping LLM apps on topic and safe with developer-defined rules	Chit-chat and banking NLU sets (76–77 intents, about 230 test samples each)	Colang rails: user message mapped to a canonical form by embedding similarity plus few-shot LLM, then a dialogue flow decides the next step	Same LLMs with plain prompting	Canonical form and next step accuracy; harmful blocked vs. helpful allowed for moderation rails	Topical rails work even with 7B–13B open models; a similarity match beats exact match for canonical forms	Topics are a flat list matched by similarity, which is my baseline R. No inheritance, no consistency check.
6	CantTalkAbout-This, Findings EMNLP 2024 [6] (P)	LLMs answering off-topic turns in task-oriented chat	1,080 synthetic dialogues, 9 domains, with distractor turns; plus 20 banking dialogues with human-written distractors	Fine-tune on the dataset so the model refuses distractors	GPT-4-turbo, GPT-3.5-turbo, Mixtral-Instruct, untuned base models	Precision, recall, F1 on distractor and on-topic turns	Fine-tuned models reach F1 0.84–0.89 on human distractors vs. 0.68 for GPT-4-turbo; all models drop on human-written distractors	Scope lives inside the model weights; no external spec to check. Its human distractors are a model for my hard out-of-scope cases.
7	EnSToM, Findings ACL 2025 [7] (P)	Small open LLMs failing to refuse off-topic turns in service chatbots	CantTalkAbout-This banking domain: 100 samples to build steering vectors, 550 distractor and 550 on-topic test samples; GPT-4o judges refuse vs. engage	Steering vector added at a middle layer at inference, with strength scaled by the model's output entropy; no training	Same model with prompt only; fixed-strength steering	Distractor accuracy (correct refusals) and on-topic accuracy (correct answers)	On LLaMA-2-7B-Chat, distractor accuracy 0.28 to 0.81 while on-topic stays about 0.90; similar gain on Ministral-8B	Same trade-off I measure, but the fix is inside the model and not auditable. Also shows how weak a plain prompt is on small models.
8	KAoS, IEEE Intelligent Systems 2004 [8] (P)	Policy control for software agents and Semantic Web services	None; three applications (grid computing, coalition search and rescue, semantic firewall)	Policies as OWL classes: positive/negative authorization and obligation; reasoner classifies actions, finds and resolves policy conflicts	Compared in prose to Rei and Ponder	Qualitative; disclosure query under 1 ms; scales to 1,000+ agents	OWL lets the same reasoner do policy lookup, conflict detection, and harmonization	Pre-LLM agents, never compared with prompting. The approach mine builds on; I take its point that OWL-DL alone could not express some policies.

TABLE II

BASELINE ERROR RATES ON THE 42-QUESTION PILOT SET (31 IN-SCOPE, 11 OUT-OF-SCOPE). CONCEPT ACCURACY IS THE SHARE OF IN-SCOPE QUESTIONS MATCHED TO THE CORRECT CONCEPT, NOT JUST A PERMITTED ONE.

Run	Out-of-scope allowed	In-scope refused	Concept acc.
Flat, hand-written descr., t=0.55	5/11	0/31	30/31
Flat, taxonomy descr., t=0.55	6/11	1/31	18/31
Flat, hand-written descr., t=0.65	5/11	1/31	30/31
Prompt, majority of 3 runs	1/11	6/31	–

TABLE III

ONTOLOGY RUNS ON THE 42-QUESTION PILOT SET. FLAT LIST ROW REPEATED FROM TABLE II FOR COMPARISON. CONCEPT ACCURACY IN PARENTHESES COUNTS A HAZARDOUS SUBCLASS OF THE CORRECT CONCEPT AS CORRECT.

Run	Out-of-scope allowed	In-scope refused	Concept acc.
Flat, taxonomy descr., t=0.55	6/11	1/31	18/31
C: ontology, matched	5/11	3/31	13/31 (18/31)
D: ontology, gold concept	0/11	0/31	31/31

TABLE IV

TEST SPLIT, 154 QUESTIONS (105 IN-SCOPE, 49 OUT-OF-SCOPE). A AND B ARE THE MAJORITY OF THREE RUNS; FLIPPED = VERDICTS THAT DIFFERED BETWEEN RUNS. CONCEPT ACCURACY COUNTS A HAZARDOUS SUBCLASS OF THE CORRECT CONCEPT AS CORRECT (IT MAKES NO DIFFERENCE ON THIS SPLIT).

Approach	OOS allowed	In-scope refused	Concept acc.	Flipped
A prose prompt	14/49	18/105	–	14
B prompt + blocklist	6/49	28/105	–	11
R flat list	39/49	8/105	47/105	–
C ontology, embedding	38/49	7/105	44/105	–
C ontology, LLM match	8/49	1/105	96/105	–
D ontology, gold	0/49	0/105	105/105	–